

Programação Orientada a Objetos

Entidades e Encapsulamento no GestorOS

[Baixe aqui os exemplos da aula](#)



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Atributo e Identidade

Uma entidade de domínio carrega identidade própria, representada aqui pelo campo id: dois clientes de mesmo nome continuam sendo objetos distintos, com vidas separadas dentro do sistema. Nome, telefone, tipo e preço não têm essa autonomia — existem apenas enquanto descrevem uma entidade específica. Cliente, Equipamento e Peça são entidades; os demais campos são atributos subordinados a elas.



Encapsulamento Além do Atributo Privado

Declarar um atributo `private` impede o acesso direto de fora da classe, mas isso sozinho não impede que o objeto assuma um valor absurdo: nada em `private String nome` garante ao menos três caracteres. O encapsulamento efetivo nasce quando o único caminho de alteração, o `setter`, recusa qualquer valor que viole a regra do domínio antes de gravar o campo.



O Construtor que Delega ao Setter

Duplicar a validação no construtor e no setter cria dois lugares para manter sincronizados, e a experiência mostra que um deles é esquecido na próxima alteração. Cliente resolve isso fazendo o construtor chamar `setNome` e `setTelefone` em vez de atribuir os campos diretamente: a regra de negócio existe em um único ponto do código, reaproveitado por qualquer alteração futura.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

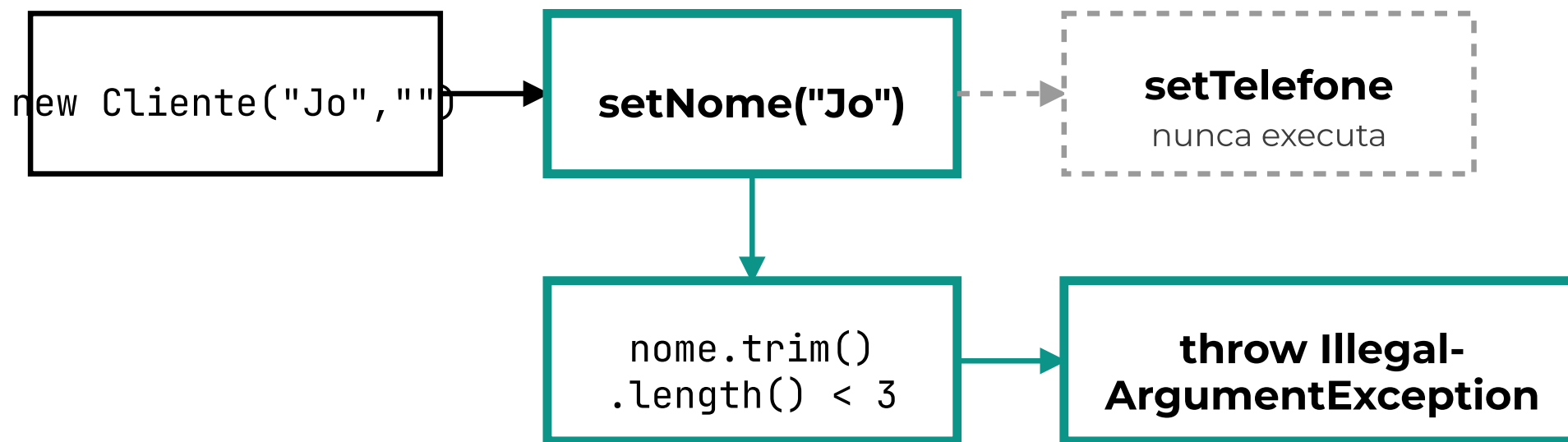
 <https://www.linkedin.com/in/everaldoribeiroads>

Por que os Setters são final

Uma subclasse pode sobrescrever um método não final, e o construtor da superclasse, ao chamar esse método antes de a subclasse terminar de inicializar seus próprios campos, executaria uma versão ainda não pronta para rodar. Marcar `setNome` e `setTelefone` como final impede a sobrescrita e garante que a validação chamada pelo construtor seja sempre a original, prevista pela classe.



Construção Interrompida por Exceção



A exceção interrompe o construtor: `setTelefone` nunca roda, e o objeto **não chega a existir**.



IllegalArgumentExceção como Linha de Defesa

Toda validação nos setters de Cliente, Equipamento e Peça termina, em caso de falha, lançando `IllegalArgumentException`. A exceção interrompe imediatamente a execução do construtor: nenhuma linha seguinte roda, nenhum campo recebe o valor recusado, e o objeto simplesmente não passa a existir. É a linguagem, não uma verificação externa, que barra o dado inválido na origem.



BigDecimal em vez de double para Dinheiro

O tipo `double` representa números em base binária, incapaz de armazenar com exatidão frações decimais comuns como 0,10, o que acumula erro em somas sucessivas de valores monetários. Peca declara o preço como `BigDecimal`, tipo que representa dígitos decimais com exatidão, e ainda oferece `signum()` para verificar o sinal do valor antes de aceitá-lo como preço válido.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Estado Sempre Válido como Contrato

Uma classe bem encapsulada assume um compromisso simples: qualquer instância acessível ao restante do programa está em estado válido, porque não existe caminho de construção ou alteração que escape da validação. Esse contrato permite que todo código que recebe um Cliente confie no nome e no telefone sem revalidar nada, pois a garantia já foi dada na origem do objeto.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Equipamento e o Vínculo por Identificador

Equipamento também valida tipo e defeito como obrigatórios, seguindo o mesmo padrão de Cliente. O vínculo com o dono do aparelho, porém, ainda é feito pelo inteiro idCliente, não por uma referência ao objeto Cliente. Essa escolha é temporária: substituir o número por uma associação direta entre objetos é o assunto que abre a próxima aula.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Setter Ingênuo x Setter que Valida

sem validação (hipotético)

setNome(nome)

`this.nome = nome;`

aceita null, "" ou "a"

setNome de Cliente (real)

setNome(nome)

`nome == null ||
length() < 3 ?`

**throw Illegal-
ArgumentException**

À esquerda o setter só atribui: qualquer valor passa. À direita, o setter de Cliente **interrompe a atribuição antes que ela aconteça.**



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Cliente: Construtor Delega ao Setter

```
public Cliente(String nome, String telefone) {  
    // O construtor delega ao setter: a regra fica escrita em um lugar so.  
    setNome(nome);  
    setTelefone(telefone);  
}  
  
public final void setNome(String nome) {  
    if (nome == null || nome.trim().length() < 3) {  
        throw new IllegalArgumentException("Nome deve ter ao menos 3 caracteres.");  
    }  
    this.nome = nome.trim();  
}
```



Peca: BigDecimal e Preço Nunca Negativo

```
private BigDecimal preco;

public Peca(String nome, BigDecimal preco) {
    setNome(nome);
    setPreco(preco);
}

public final void setPreco(BigDecimal preco) {
    // Invariante do dominio: preco nunca e negativo.
    if (preco == null || preco.signum() < 0) {
        throw new IllegalArgumentException("Preco nao pode ser negativo.");
    }
    this.preco = preco;
}
```



GestorOS: o Objeto que Não Nasce

```
public static void main(String[] args) {  
    Cliente cliente = new Cliente("Marina Alvarenga", "(32) 98811-4402");  
    cliente.setId(1);  
    // Equipamento e Peca tambem sao criados aqui; omitidos por espaco.  
  
    // O objeto invalido nao nasce: a execucao interrompe a construcao.  
    try {  
        new Cliente("Jo", "");  
    } catch (IllegalArgumentException e) {  
        System.out.println("Recusado ...: " + e.getMessage());  
    }  
}
```



Atividades

Exercício 1

Por que o construtor de Cliente chama setNome e setTelefone em vez de atribuir os campos diretamente?

Exercício 2

Por que setPreco de Peca usa BigDecimal em vez de double? O que aconteceria numa soma de vários preços com double?

Exercício 3

Por que os setters de Cliente são declarados final? O que poderia dar errado se não fossem?



Desafio extra

Opcional

Quem quiser ir além do combinado nesta semana:

1. Adicionar validação de e-mail em Cliente, com um novo setter final que segue o mesmo padrão.
2. Criar a classe OrdemServico no pacote model, com sua própria validação de invariantes.

Nada disso é cobrado na entrega. O projeto segue completo sem essas adições.

